Universidad Nacional
Autónoma de México

Facultad de Ciencias

Proyecto Final: Agente que juegue Snake usando redes neuronales

Alumnos:
Mijail Aron Álvarez Cerrillo
Alejandro Iabin Arteaga Hernández
José Fernando Reyes García
Alejandro Emmanuel Vargas Aldaco

Profesor:
Gustavo De la Cruz Martínez

Ayudantes:
Estefanía Prieto Larios
Osvaldo Baruch Acevedo Badillo
Rodrígo Eduardo Colín Rivera

Resumen
24 de Noviembre, 2019

Introducción

Una de las ramas mas destacadas de la Inteligencia Artificial es la que corresponde a las Redes Neuronales, en el entendido de que son aquellas en cuales existen elementos procesadores de información de cuyas interacciones locales depende el comportamiento del conjunto del sistema.

Las redes neuronales tratan de emular el comportamiento del cerebro humano, el cual se caracteriza por el aprendizaje a través de la experiencia y extracción de conocimiento genérico a partir de un conjunto de datos ya conocidos

Por lo tanto, podemos decir que las características fundamentales de las redes neuronales son:

- **Aprenden de la experiencia:** Las redes neuronales pueden modificar su comportamiento como respuesta a su entorno. Dado un conjunto de entradas (quizá con las salidas deseadas), las redes neuronales se ajustan para producir respuestas consistentes. Una amplia variedad de algoritmos de entrenamiento se han desarrollado, cada uno con sus propias ventajas e inconvenientes.
- **Generalizan de ejemplos anteriores a los ejemplos nuevos:** Una vez que la redes neuronales esté entrenada, la respuesta de la red puede ser, hasta un cierto punto, insensible a pequeñas variaciones en las entradas, lo que las hace idóneas para el reconocimiento de patrones.
- **Abstracción de la esencia de las entradas:** Algunas redes neuronales son capaces de abstraer información de un conjunto de entradas. Por ejemplo, en el caso de reconocimiento de patrones, una red puede ser entrenada en una secuencia de patrones distorsionados de una letra. Una vez que la red sea correctamente entrenada será capaz de producir un resultado correcto ante una entrada distorsionada, lo que significa que ha sido capaz de aprender algo que nunca había visto.

Snake

En la década de los 70's fue lanzado al mundo uno de los videojuegos que más alcanzaría popularidad alrededor de todo el mundo, el cual le debe gran audiencia a la marca Nokia, por incluirlo en uno de sus más populares celulares. Se trata de el juego de Snake.

El argumento del juego, es simple: Controlas una serpiente que esta encerrada en una celda de nxm cuadros, y dicha serpiente, tiene de ancho 1 cuadro, y se extiende a lo largo en l cuadros, o sea, su tamaño es de $lx1$ cuadros. Por lo tanto, su desplazamiento es de cuadro por cuadro, avanzando por su cabeza y arrastrando su cola.

Dentro de la celda, caerán aleatoriamente manzanas, o algún alimento para dicha serpiente. El objetivo del jugador pues, es conducirla hasta el alimento, procurando evitar que esta choque, ya sea con una pared, o con su propio cuerpo, el cual, aumentara de

tamaño en un cuadro, cada vez que sea alimentada.

Es un juego de dinámica simple, pero que ha logrado captar la atención de los jugadores, manteniendo su popularidad aún hoy en día.

Objetivo del proyecto

El objetivo de este proyecto a realizar, es simular a un agente que tenga por fin jugar una partida de Snake, intentado que sea lo más parecido al juego que realizaría un ser humano, esto último, se pretende conseguir con el apoyo de redes neuronales.

Descripción del Agente

El fin de este agente es jugar Snake valiéndose de una red neuronal que este equipo implemente.

Para desarrollar una red neuronal para este caso se necesitan usualmente los siguientes 3 pasos:

1. Generación de datos de entrenamiento.
2. Entrenamiento de la red neuronal.
3. Pruebas

Este conjunto de datos se vera representado como una pareja de tuplas, la primera con 7 valores y la segunda con 3, estando en relación de entrada - salida, o sea, la primer tupla es la entrada que se recibe y la segunda es el movimiento que hace dada esa entrada. La estructura de las tuplas esta hecha a fin de representar la información de la siguiente manera.

Para los datos de entrada, están representados:

1. Sobre el cuadro que esta a la izquierda de la cabeza, si esa bloqueado o no (1 o 0).
2. Sobre el cuadro que esta enfrente, si esa bloqueado o no (1 o 0).
3. Sobre el cuadro que esta a la derecha, si esa bloqueado o no (1 o 0).
4. Valor X de la posición del alimento de la serpiente.
5. Valor Y de la posición del alimento de la serpiente.
6. Valor X de la posición actual de la serpiente (cabeza).
7. Valor Y de la posición actual de la serpiente (cabeza).

La razón por la que se utilizaron estos datos de entrada y no el tablero completo como se había propuesto al principio, es que debido a la enorme complejidad combinatoria del tablero entero, para que la red comenzara a percibir aprendizaje significativo, se necesitaba una enorme cantidad de datos de entrenamiento, y ya que el objetivo final era

que simulara el comportamiento humano, necesitábamos una ingente cantidad de datos generados por humanos, lo cual es muy difícil de obtener, creemos que los datos utilizados en la entrada son suficientes para un aprendizaje adecuado sin demasiadas variables

Por cuestiones de tiempo y disponibilidad, el conjunto de datos de entrenamiento no se genera por juegos propios de los integrantes del equipo. En su lugar, y para facilitar el desarrollo del conjunto para entrenamiento se tiene un método, que calcula el ángulo entre la posición de la serpiente y el alimento, por lo que le indica cual dirección seguir a fin de llegar al alimento.

Para los datos de salida, sencillamente es:

1. (1,0,0) la serpiente gira a la izquierda.
2. (0,1,0) la serpiente se mantiene en la misma dirección.
3. (0,0,1) la serpiente gira a la derecha.

La arquitectura de la red neuronal son 4 capas densas la primera es la capa de entrada con 7 nodos, las 2 siguientes son las capas intermedias, cada una con 64 nodos y activación relu. Y la última es la capa de salida, consta de 3 neuronas y con activación softmax. Entrenamos el modelo en nuestras computadoras y solo agregamos el modelo ya entrenado, pero para ver la arquitectura de la red, agregamos también el archivo de python con la red y los archivos de entrenamiento.

El modelo queda de la siguiente manera:

```
"""Creo la red neuronal, empieza con una entrada secuencial """
model = Sequential()
model.add(Dense(units=9,input_dim=7))
#2 capas densas de 64 nodos y activacion relu
model.add(Dense(units=64, activation='relu'))
model.add(Dense(units=64, activation='relu'))
#Una capa de salida de 3 nodos, con el siguiente movimiento
model.add(Dense(output_dim=3,  activation = 'softmax'))


#Compilamos el modelo
model.compile(loss='mean_squared_error', optimizer='adam', metrics=['accuracy'])
#Lo entrenamos
model.fit((np.array(training_data_x).reshape(-1,7)),( np.array(training_data_y).reshape(-1,3)), batch_size = 256,epochs= 100)
```

Se hicieron 100 iteraciones de entrenamiento y se utilizó la función adam como función de aprendizaje.

Tipo de Agente

El tipo de agente ideal para atacar este problema es del tipo que aprende. La forma en que trabaja se puede ver en la siguiente figura:

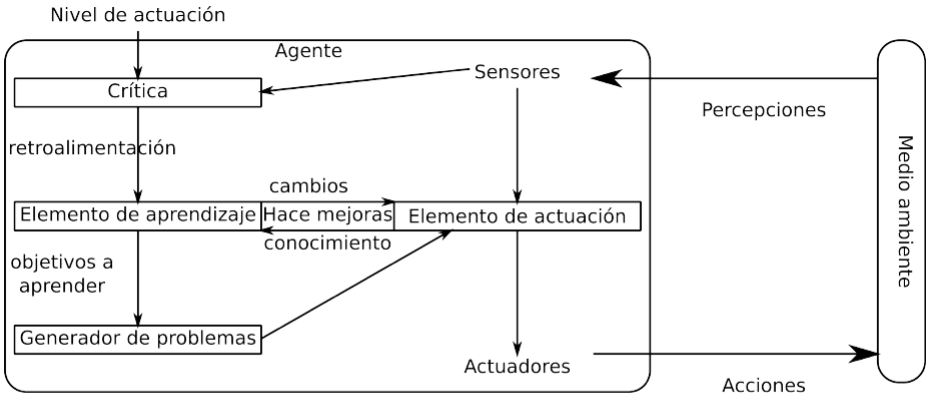

Figura: Modelo general para agentes que aprenden [Russell & Norving (2004)].

Justo el objetivo es que la red neuronal aprenda de cada dato que obtiene como es mejor jugar.

Rendimiento

La medida de rendimiento para determinar la efectividad del agente será el numero de alimentos capaz de tomar antes de chocar y por ende, terminar el juego. O bien, de la misma manera, la longitud de la serpiente al momento de un choque.

Sensores

Los datos de entrada para el agente son los datos previamente mencionados, que van conformando el conjunto de datos de entrenamiento para la red neuronal implementada. Estos datos deben estar cargados previos a una partida. Al momento de jugar, los datos de entrada son su posicion (x_s, y_s), y la posicion del alimento (x_a, y_a)

Actuadores

Son las instrucciones de movimiento que hará sobre la serpiente, que son simplemente "Girar izquierda", "Girar derechaz "Seguir adelante".

Entorno

El entorno en el que trabaja es una celda de nxm cuadros, en los cuales se puede desplazar siempre y cuando no choque con algo. Por lo que se puede afirmar que el entorno es **discreto**. Es de notar que se trata de un único agente que interactúa, por lo que se trata de un **agente individual**

Es **parcialmente observable**, pues puede conocer su ubicación en el tablero con vectores de posición (x, y), así como la posición del alimento hacia donde se debe dirigir,

pero no tiene noción de donde está la totalidad de su cuerpo.

Cada que recolecta algún alimento, el entorno se modifica, pues se agrega otro alimento en otra posición aleatoria. Por lo tanto, es **secuencial**, y dado que se coloca aleatoriamente el alimento, es **estocástico**.

En resumen, el REAS para este agente queda modelado de esta forma.

Agente	Rendimiento	Entorno	Actuadores	Sensores
Agente que juega Snake	Longitud de la serpiente hasta el momento de chocar o número de alimentos recolectados antes de choque.	Celda de nxm cuadros, donde uno de ellos tiene un alimento, el cual es el objetivo a conseguir.	Giro a la izquierda, giro a la derecha y seguir adelante	Dataset de entrenamiento para red neuronal, posiciones de la serpiente y del alimento objetivo.

Conclusiones

La red neuronal propuesta, aunque pueda parecer simple, tiene un gran rendimiento, logra imitar de manera muy parecida el comportamiento determinista con el cual fue entrenado, el siguiente paso es generar un dataset que contenga una gran cantidad de juegos humanos, para corroborar su comportamiento con movimientos no deterministas.

Referencias

- Raquel Flórez López, José Miguel Fernández Fernández, 2008, Las redes neuronales artificiales, Fundamentos teoricos y aplicaciones practicas.
- https://es.wikipedia.org/wiki/Red_neuronal_artificial
- https://www.frro.utn.edu.ar/repositorio/catedras/quimica/5_anio/orientadora1/monograias/matich-redesneuronales.pdf
- Notas de Veronica Esther Arriola Rios https://sites.google.com/a/ciencias.unam.mx/redesneuronales/apuntes
- https://theailearner.com/2018/04/19/snake-game-with-deep-learning/